

A decorative vertical bar on the left side of the slide, consisting of a dark brown bar and a red arrow pointing right.

CS1002 – Programming Fundamentals

Lecture # 16
Tuesday, October 18, 2022
FALL 2022
FAST – NUCES, Faisalabad Campus

Rizwan Ul Haq

2 Tracing an algorithm

- The current values of algorithm variables at various points during execution can be known by tracing the algorithm with a table called Trace Table
- A technique used to detect any possible logical errors
- A typical trace table looks like:

Step / Instruction #	Variable 1	Variable 2	Variable ...
1	5	0	...
2	6	2	...
...	...	...	...

3

Example -

- Create & fill a trace table for the following pseudo code.

```
1. Y = 3
2. X = 1
3. While X < 4
  3.1 Y = Y + X
  3.2 X = X + 1
4. Stop
```

Step #	X	Y	X < 4 ?

Example -

- Create & fill a trace table for the following pseudo code

```

1. Y = 3
2. X = 1
3. While X < 4
    3.1 Y = Y + X
    3.2 X = X + 1
4. Stop
  
```

Step #	X	Y	X < 4 ?
1	-	3	
2	1	3	
3	1	3	T
3.1	1	4	
3.2	2	4	
3	2	4	T
3.1	2	6	
3.2	3	6	
3	3	6	T
3.1	3	9	
3.2	4	9	
3	4	9	F
4	4	9	

do...while Looping (Repetition) Structure

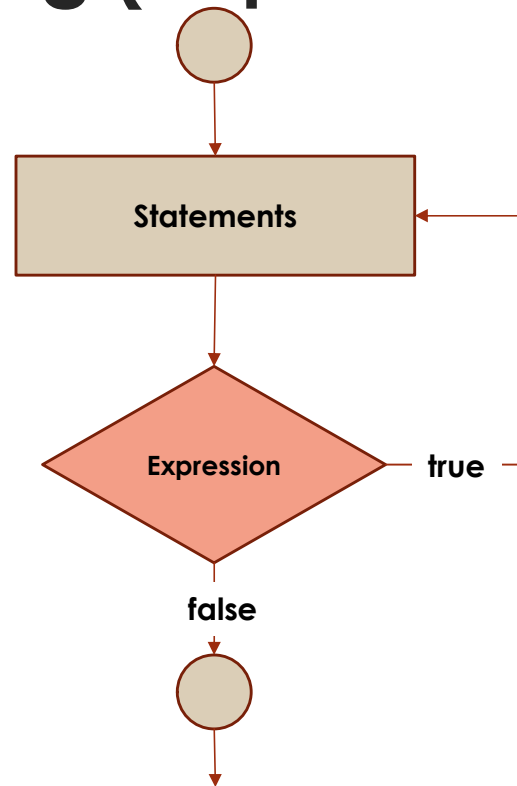
- General form of a **do...while**:

```
do  
    statement  
while (expression);
```

- The statement executes first, and then the expression is evaluated
- To avoid an infinite loop, body must contain a statement that makes the expression false
- The statement can be simple or compound
- This loop is **post-test** loop
- Loop always iterates at least once

6

do...while Looping (Repetition) Structure (cont'd.)



do-while loop

do...while Looping (Repetition) Structure

```
number = 0;
do
{
    cout << number << " " ;
    number = number + 5 ;
}
while ( number <= 20) ;
```

The output of this code is:

0 5 10 15 20

After 20 is output, the statement:

number = number+5

Changes the value of number to 25 so number <= 25 becomes **false**, which stops the loop

do...while Looping (Repetition) Structure

Consider the following two loops:

```
a. i = 11;
   while(i <= 10)
   {
       cout << i << " " ;
       i = i + 5 ;
   }
   cout << endl ;
```

```
b. i = 11 ;
   do
   {
       cout << i << " " ;
       i = i + 5 ;
   }
   while ( i <= 10);
   cout << endl;
```

In (a), the `while` loop produces nothing. In (b), the `do...while` loop outputs the number 11 and also changes the value of `i` to 16

do...while Looping (Repetition) Structure

```
int score ;  
do  
{  
    cout << "Enter a score between 0 and 50: " ;  
    cin >> score ;  
    cout << endl ;  
}while (score > 0 && score < 50);
```

Example 5-20: Divisibility Test by 3 and 9

```
sum = 0;
temp = num;
do
{
    sum = sum + num % 10 ;    //extract the last digit
                             //and add it to sum
    num /= 10 ;              //remove the last digit
}while(num > 0);

cout << "The sum of the digits = " << sum << endl ;

if(sum % 3 == 0)
    cout << temp << "is divisible by 3" << endl ;
else
    cout << temp << " is not divisible by 3 " << endl ;

if (sum % 9 == 0)
    cout << temp << "is divisible by 9" << endl ;
else
    cout << temp << " is not divisible by 9" << endl ;
```

Choosing the Right Looping Structure

- All three loops have their place in C++
 - If you know or can determine in advance the **number of repetitions needed**, the **for** loop is the correct choice
 - If you do not know and **cannot determine in advance** the number of repetitions needed, and it could be zero, use a **while** loop
 - If you do not know and cannot determine in advance the number of repetitions needed, and it is **at least one**, use a **do...while** loop

break and continue Statements

- **break** and **continue** alter the flow of control
- **break** statement is used for two purposes:
 - To exit early from a loop
 - Can eliminate the use of certain (flag) variables
 - To skip the remainder of the switch structure
- After the **break** statement executes, the program continues with the first statement after the structure

break and continue Statements

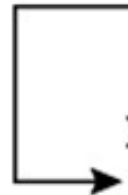
- **continue** is used in **while**, **for**, and **do...while** structures
- When executed in a loop
 - It skips remaining statements and proceeds with the next iteration of the loop

break Statement

```

while (test expression) {
    statement/s
    if (test expression) {
        break;
    }
    statement/s
}

```



```

do {
    statement/s
    if (test expression) {
        break;
    }
    statement/s
}
while (test expression);

```



```

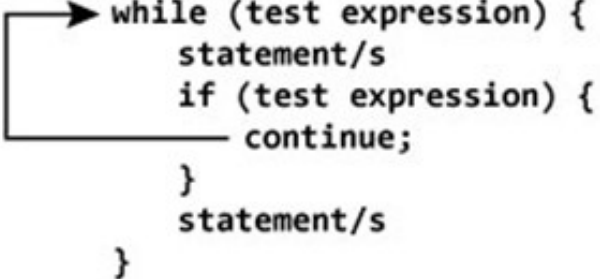
for (initial expression; test expression; update expression) {
    statement/s
    if (test expression) {
        break;
    }
    statements/
}

```

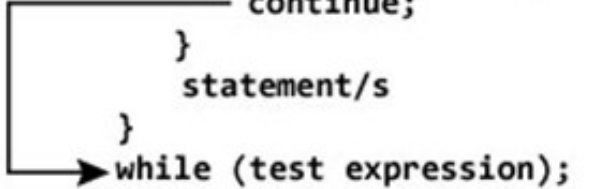


15 continue Statement

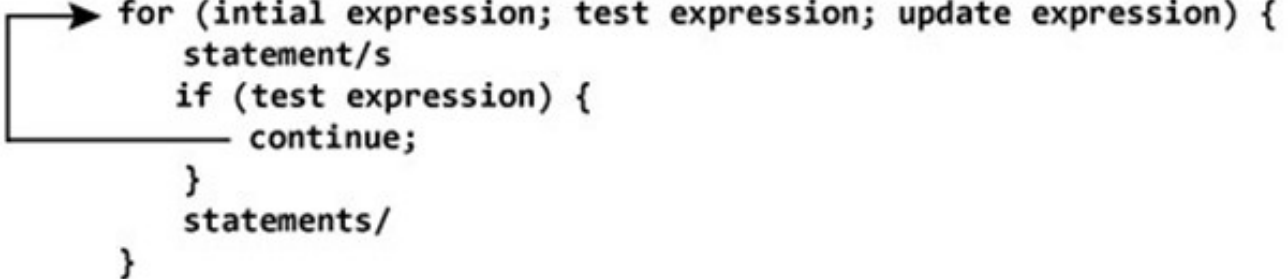
```
while (test expression) {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statement/s  
}
```



```
do {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statement/s  
} while (test expression);
```



```
for (initial expression; test expression; update expression) {  
    statement/s  
    if (test expression) {  
        continue;  
    }  
    statements/  
}
```



Example

```
int sum = 0;
bool isNegative = false;
int num ;

cin >> num;

while (cin && !isNegative)
{
    if(num < 0)    //if num is negative, terminate the loop
                  //after this iteration
    {
        cout << "Negative number detected " << endl;
        isNegative = true;
    }
    else
    {
        sum = sum + num;
        cin >> num;
    }
}
```

break Statement Example

```
int sum = 0;
int num ;

cin >> num;

while (cin)
{
    if(num < 0) //if num is negative, terminate the loop
    {
        cout << "Negative number detected " << endl;
        break ;
    }
    else
    {
        sum = sum + num;
        cin >> num;
    }
}
```

18 continue Example

```
int sum = 0;
int num ;

cin >> num;

while (cin)
{
    if(num < 0)
    {
        cout << "Negative number detected " << endl;
        cin >> num ;
        continue;
    }
    sum = sum + num;
    cin >> num;
}
```

19 continue Example

```
for (int i = 0; i < 100; ++i)
{
    // Skip all the multiples of 7
    // Operator % calculates the modulus (remainder)
    // of a division; if (i % 7) is equal to zero
    // this means that i is a multiple of 7.
    if ((i % 7) == 0)
        continue;
    cout << i << " ";
}
```